KiwiShare

测试策略

面向 KiwiShare 移动二手市场的风险驱动质量保证体系

版本	课程	负责人
0.1 - 团队评审草案	COMPSCI 734	Five Guys

依据 COMPSCI 734 Lecture 07、已批准的 KiwiShare Pitch、产品需求与模块 Backlog 编制。

策略概览

面向五人团队的可持续测试体系：每次变更获得快速反馈，在集成边界建立更强置信度，并在发布前按风险配置质量保证。

01 测试左移

在代码进入共享分支前，用快速测试发现逻辑和 UI 缺陷。

02 覆盖状态

不能只测成功：加载、空、错误、离线和权限拒绝同样重要。

03 保护信任

支付、二维码、授权、位置、聊天和举报获得最高级别保证。

04 门禁流程

Task Branch 到 pre 再到 main：质量证据随每次变更向前传递。

越靠近底层反馈越快；越靠近顶层置信度与执行成本越高



大量单元测试 + 大量 Widget 测试 + 针对性集成测试 + 少量关键 Patrol 旅程。

目录

1. 目的
2. 依据与关联文档
3. 质量目标
4. 测试范围
5. 测试原则
6. 基于风险的优先级
7. 测试层级
8. 关键旅程覆盖矩阵
9. 状态与失败覆盖
10. 测试环境与数据
11. 第三方与原生能力测试策略
12. 非功能测试
13. 设备与平台覆盖
14. 自动化与分支质量门禁
15. 覆盖率与质量目标
16. 缺陷分级
17. 准入与退出标准
18. 可追溯性、命名与证据
19. 责任与维护
20. CI 强制执行前的待决事项
21. 初始实施顺序
附录 A-C

建议阅读顺序：产品与工程相关方应先阅读第 3、6、8、14 和 17 节；负责实现测试的贡献者还应阅读第 5、7、9、10 和 11 节。

字段	内容
文档负责人	Five Guys
产品	KiwiShare
课程	COMPSCI 734 - Mobile, Web & Enterprise Computing
版本	0.1（团队评审草案）
状态	待批准
最后更新	2026 年 8 月 15 日
主要集成分支	`pre`

1. 目的

本文档规定 KiwiShare 团队在开发全过程及发布前验证产品质量的方法。它将产品风险、系统架构与课程指导转化为一套五人团队能够持续执行、重复验证的测试流程。

本策略旨在：

- 尽早、以较低成本发现缺陷；
- 保护关键的市场交易与履约流程；
- 验证加载、成功、空数据、错误、离线和权限拒绝等状态；
- 对每次变更提供快速反馈，并在合并或发布前提供更强的质量保证；
- 使质量证据能够追溯至产品需求和 GitHub Issue；
- 减少对缓慢且不一致的人工回归测试的依赖。

本文档是测试策略，不是单个测试用例的目录。详细测试用例应存放在测试代码、Issue 验收标准以及发布/UAT 检查清单中。

2. 依据与关联文档

本策略依据以下资料制定：

- `CS734 Lecture 07 - Testing Mobile Applications`；
- `COMPSCI 734_Five Guys_KiwiShare Elevator Pitch - Assignment 1`；
- 已批准的 KiwiShare 产品需求；
- 当前 KiwiShare 模块 Backlog 与 Issue 任务拆解。

本文档应与以下资料共同阅读：

- `docs/product-requirements/product-requirements.md`；
- `docs/architecture.md`；
- `docs/api-spec.md`；
- `docs/schemas/`；
- `docs/security-owasp.md`；
- `docs/design-guidelines/`；
- `docs/gitlab-flow.md`；
- `docs/deployment.md`。

若本文档与已批准的产品需求、API 契约、数据 Schema 或安全控制发生冲突，团队必须在开始实现前提出并解决该冲突。

3. 质量目标

只有当团队能够证明 KiwiShare 满足以下条件时，产品才可视为可测试并达到发布准备状态：

1. 功能正确 - 必需的用户旅程和业务规则按照规格运行。
2. 安全可信 - 身份、授权、举报、支付、订单核验和信誉控制在正常使用或对抗性操作下均不可被绕过。
3. 可靠可恢复 - 网络、后端、权限、通知或第三方服务失败时，系统产生受控且可恢复的结果。
4. 无障碍且易用 - 核心页面通过点击目标、语义标签和文字对比度自动检查，并在真实设备条件下保持可用。
5. 保护隐私 - 个人、位置、聊天、举报及支付相关数据仅向获授权主体开放。
6. 可维护 - 测试应确定、可读、相互隔离，并足够快速以便日常执行。

4. 测试范围

4.1 范围内

本策略覆盖已纳入 MVP 实现范围的以下产品界面和能力：

- Flutter 移动应用；
- 身份认证与个人资料管理；
- 商品发现、搜索、筛选、排序与商品详情；
- 商品发布、图片上传和 AI 辅助描述；
- 关注列表与价格变动通知；
- 上下文聊天、文本净化、图片/语音附件和推送通知；
- 位置权限、附近商品和安全见面地点建议；
- 交易、支付、订单、二维码核验、完成、取消与退款流程；
- 举报、屏蔽、内容治理、评分与用户信用；
- Firebase 托管服务及安全规则；
- 具有特权的 Node.js 服务/Cloud Functions 和数据持久化；
- 若纳入发布范围，则包括 React 管理后台功能；
- CI 检查、测试环境与发布质量证据。

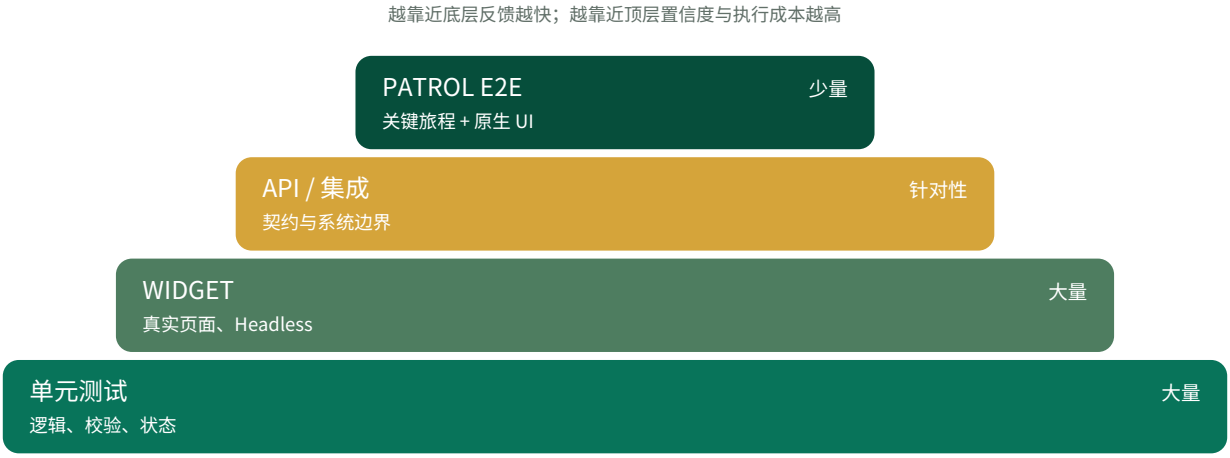
4.2 初始课程版本范围外

除非另行批准，首个版本不强制要求：

- 大规模真实设备农场；
- 穷尽测试所有 Android 与 iOS 型号/版本组合；
- 全规模生产负载测试；
- 对每个页面执行像素级 Golden Test；
- 由外部安全机构实施渗透测试；
- 验证 Google、Firebase、Stripe、FCM、地图、社交分享或 AI 供应商平台的内部正确性。

对于外部服务，团队采用契约验证、官方 Emulator/Test Mode、受控 Fake，以及少量端到端集成检查进行覆盖。

5. 测试原则



将课程原则落实到 KiwiShare：在金字塔底层最大化快速反馈，把成本较高的原生 E2E 测试留给最关键的用户旅程。

5.1 遵循移动应用测试金字塔

KiwiShare 应维护：

- 大量单元测试：覆盖业务逻辑、校验、状态转换和 ViewModel；
- 大量 Widget/组件测试：覆盖页面、交互、UI 状态和无障碍；
- 适量 API/集成测试：验证组件与服务之间的边界；
- 少量端到端测试：覆盖演示或发布时不可接受失败的关键旅程。

发现缺陷后，默认应在能够稳定复现该缺陷的最低测试层级增加回归测试。

5.2 测试可观察行为，而非实现细节

测试应验证输入、输出、状态、用户可见行为、契约和经授权的副作用。测试不应依赖私有方法、任意 Widget Tree 结构或偶然的实现选择。

5.3 面向可测试性设计

网络客户端、Repository、时钟、ID 生成器、存储适配器、通知网关、位置 Provider、支付网关和 AI 服务必须支持依赖注入。单元测试和 Widget 测试不得调用真实网络。

5.4 保证测试确定性

自动化测试不得依赖：

- 生产服务或生产数据；
- 开发者手工启动后端；
- 真实支付卡或个人账号；
- 不受控的系统时间、随机值或实时 GPS 坐标；
- 执行顺序或其他测试遗留的状态。

5.5 自动化重复检查，以人工测试提供判断

自动化用于保护稳定且可重复的预期。可用性、视觉质量、模糊需求和真实用户反馈仍需人工探索测试与用户验收测试（UAT）；但人工测试不能替代自动化回归测试。

6. 基于风险的优先级

P0	严重	资金、授权、隐私、交易完整性
P1	高	核心市场功能与信任旅程
P2	中	体验下降，但存在替代方案
P3	低	外观或低影响行为

测试投入按照影响与发生可能性分配，而不是平均覆盖所有功能。

优先级	含义	KiwiShare 示例	必需的质量保证
P0 - 严重	可能造成经济损失、未授权访问、隐私暴露、交易状态损坏或核心产品不可用	授权、支付、退款、订单归属、二维码核验、重复请求、私密位置/聊天/举报数据	单元 + 边界集成 + E2E 主流程及关键失败路径
P1 - 高	阻断主要市场流程或严重损害用户信任	登录、发布商品、商品读取、搜索、聊天、举报/屏蔽、评分/信用、位置权限	单元 + Widget + 集成；选择性 E2E
P2 - 中	降低体验但存在替代方案	关注列表、推送通知、深色模式、社交分享、AI 建议	单元/Widget 与针对性集成
P3 - 低	外观或低影响行为	非关键动画、装饰性布局、应用图标	视情况进行 Widget/人工视觉检查

任何 P0 功能均不得只依赖人工 Happy Path 演示。

7. 测试层级

7.1 单元测试

目的： 在不使用真实 UI、设备、网络或外部服务的情况下，验证单个函数、类、服务、Reducer、Controller 或 ViewModel。

预期数量： 大量。

Flutter/Dart 范围：

- 字段校验与表单规则；
- 商品搜索、筛选和排序逻辑；
- 价格与分类校验；
- 商品发布 ViewModel 状态；
- 权限和位置降级决策；
- 聊天文本净化与消息状态逻辑；
- 关注列表变更和通知资格；
- 订单与二维码状态转换规则；

- 举报、评分和信用计算；
- 将 API/Domain 错误映射为安全的用户提示；
- Repository/Service 调用次数及参数符合预期。

Node.js/后端范围：

- 请求校验与规范化；
- 授权与资源归属检查；
- 幂等性和重复事件保护；
- 允许与拒绝的状态转换；
- 使用 Fake Gateway 验证支付/退款编排逻辑；
- 二维码生成与核验规则；
- 内容治理与文本净化；
- 通知资格和 Payload 构造；
- 数据访问与错误映射。

React 范围（如适用）：

- 状态与表单逻辑；
- 由授权状态控制的操作；
- 内容治理和商品管理行为。

方法：

- 在 Dart 的可注入边界使用手写 Fake 或 `mocktail`；
- Node/React 使用团队批准的 Mock 框架；
- 使用固定时钟、确定性 ID 和显式 Fixture；
- 按风险验证成功、无效输入、缺失数据、未授权访问、依赖失败以及重试/幂等路径。

位置与命名：

- Flutter 测试位于 `test/`，文件名以 `_test.dart` 结尾；
- 后端和 Web 测试遵循对应 Package 的约定；
- 测试名称应描述行为，例如：`rejects completion when only one party has verified the QR code`。

7.2 Flutter Widget 测试

目的： 以 Headless 方式渲染真实 Flutter Widget，并以接近单元测试的速度验证完整页面行为。

预期数量： 大量；每个核心页面均应具有 Widget 覆盖。

每个核心页面应按适用情况证明：

- 加载状态；
- 有数据/成功状态；
- 空状态；
- 校验状态；
- 可恢复错误状态；
- 离线/降级状态；
- 已授权与未授权操作状态；

- 导航与主要交互；
- 所需的浅色/深色主题行为；
- 无障碍规范。

示例包括：

- 登录表单校验与错误显示；
- 商品列表、无结果、排序/筛选以及进入详情页；
- 商品详情页仅对 Owner 显示编辑/删除控件；
- 发布表单校验、上传进度与失败恢复；
- 聊天列表/记录、发送消息、附件错误和举报操作；
- 个人资料、信用分、已发布/已售商品状态与退出登录；
- 加入/移除关注列表和价格变动提示；
- 使用 Fake Repository 的支付与二维码状态页面。

实现指导：

- 使用注入的 Fake Repository/Service，Pump 真实 App 或页面 Shell；
- 使用 `find.text`、`find.byType`、`find.byIcon` 以及稳定的 `Key`；
- 使用 `tester.tap`、`enterText`、`drag` 等交互 API；
- 断言中间状态/加载帧时使用 `pump()`；
- 仅当 Widget Tree 确实会稳定时使用 `pumpAndSettle()`，不得对无限动画盲目使用；
- 优先使用语义标签和稳定 Test Key，避免脆弱的 Widget Tree 遍历。

7.3 无障碍测试

核心页面的无障碍检查属于自动化构建门禁，而非可选的人工提醒。

Widget 测试必须使用 Semantics，并按适用情况验证：

- `androidTapTargetGuideline` - 可点击目标至少为 48 x 48 dp；
- `iOSTapTargetGuideline` - 可点击目标至少为 44 x 44 pt；
- `textContrastGuideline` - 文字满足框架规定的最低对比度；
- `labeledTapTargetGuideline` - 可点击控件具有有意义的语义标签。

额外人工检查应包括：

- 大字号/文字缩放；
- 关键旅程的屏幕阅读器朗读顺序；
- 错误信息不只依赖颜色传达；
- 浅色/深色模式可读性；
- Web 管理后台的键盘焦点与导航。

7.4 API 与服务集成测试

目的： 在使用受控测试数据的前提下，验证组件跨真实边界协同工作。

预期数量： 针对性配置；多于 E2E，少于单元/Widget 测试。

覆盖范围应包括：

- API 请求/响应 Schema 与错误契约；
- 身份 Token 校验和授权规则；
- 使用到的 Firebase/Firestore 安全规则；
- Node.js 服务到数据库的行为；
- 序列化/反序列化与向后兼容契约；
- 商品新增/更新/删除及 Owner 限制；
- 聊天持久化与仅参与者访问；
- 关注列表持久化与通知触发条件；
- 订单创建、支付事件、退款与幂等性；
- 双方获授权用户的二维码核验；
- 举报生命周期与治理权限；
- 仅在符合条件的已完成交易后更新评分/信用。

测试必须使用隔离测试数据库、可行时使用 Firebase Emulator，并使用供应商批准的 Sandbox/Test Mode。严禁使用生产凭据和生产数据。

7.5 使用 Patrol 的移动端到端测试

目的： 在 Emulator/Device 上运行真实移动应用，覆盖 Flutter Widget 测试无法控制的原生平台 UI。

预期数量： 少量；每个关键用户旅程一个聚焦测试。

纯 Flutter 流程可以使用 Flutter `integration_test`，但它无法完整自动化原生权限弹窗、通知和 Platform View。对于跨越这些边界的旅程，KiwiShare 应使用 Patrol。

首批必需 Patrol 覆盖：

1. 身份认证 Smoke Journey - 启动应用，使用受控测试身份认证，验证登录状态，然后退出。
2. 商品发布 Journey - 拍摄/选择图片，输入详情，处理权限状态，提交并确认商品出现在列表中。
3. 位置 Journey - 请求位置权限，分别测试允许和拒绝路径，并验证安全降级行为。
4. 交易 Journey - 创建订单，在范围内完成 Sandbox 支付，按照批准的状态模型执行二维码核验并确认交易完成。
5. 聊天/举报 Journey - 从商品详情进入聊天，发送确定性消息，并访问举报/屏蔽流程。
6. 通知 Smoke Journey - 在 CI/设备能力允许时，触发并观察一条有代表性的推送通知流程。

如果某个旅程无法保证确定性，或每个 PR 执行成本过高，则必须明确分配到 `pre` 回归套件或发布清单，而不能静默省略。

Patrol 测试存放在 `patrol_test/`，通过 `patrol test` 执行，而不是 `flutter test`。

7.6 Web 端到端测试

若 React 管理后台纳入发布范围，团队批准的浏览器自动化工具应覆盖：

- 获授权管理员登录；
- 搜索和审查商品/举报；
- 执行允许的治理操作；

- 拒绝未授权访问；
- 治理决定和审计信息被持久化。

7.7 人工探索测试与 UAT

人工测试用于需要人类判断和探索的事项，包括：

- 不明确或刚发生变化的行为；
- 视觉完成度与设计规范符合性；
- 真实设备的相机、GPS、通知和网络行为；
- 被中断的用户旅程；
- 可用性和信任感；
- 来自代表性 UoA 用户的校园试点反馈。

UAT 必须使用源自产品需求的预定义场景。反馈应记录为 GitHub Issue，并包含环境、步骤、预期结果、实际结果、严重级别和证据。

8. 关键旅程覆盖矩阵

下表定义最低覆盖要求。详细测试用例将随实现持续演进。

旅程/能力	单元	Widget	API/集成	Patrol/E2E	人工/UAT	优先级
邮箱与 Google 身份认证	是	是	是	Smoke	是	P0/P1
个人资料与授权	是	是	是	-	是	P0
浏览、搜索、筛选、排序、商品详情	是	是	是	Smoke	是	P1
发布/编辑/删除自己的商品	是	是	是	是	是	P1
图片上传及无效/超大媒体	是	是	是	针对性	是	P1
AI 描述建议与降级	是	是	契约	-	是	P2
关注列表与价格变动通知	是	是	是	针对性	是	P1/P2
聊天、附件、净化、举报/屏蔽	是	是	是	是	是	P0/P1
位置权限与安全地点建议	是	是	是	是	是	P0/P1
交易/订单状态转换	是	是	是	是	是	P0
支付、失败、Webhook 重放、退款	是	是	是	Sandbox Smoke	是	P0
双方二维码核验与重放防护	是	是	是	是	是	P0
评分与用户信用资格	是	是	是	针对性	是	P1
滥用举报生命周期与治理	是	是	是	针对性	是	P0/P1
离线/重连与后端失败	是	是	是	针对性	是	P1
深色模式与无障碍	-	是	-	-	是	P1/P2

"针对性"表示：相关原生/平台能力发生变化时，以及 `pre` 回归阶段执行；并非每个小型 PR 都必须运行。

9. 状态与失败覆盖

每个核心旅程的测试设计必须明确考虑人工演示中容易遗漏的状态：

- 加载与延迟响应；
- 空数据或数据已删除；
- 无效或过期身份认证；
- 资源归属/角色未授权；
- 权限拒绝或永久拒绝；
- 网络超时、离线与重连；
- 后端 4xx/5xx 响应；
- 供应商/API 失败；
- 重复提交、Webhook 重放、重复扫码或双击；
- 部分完成与安全重试；
- 并发更新或版本过期；
- 与被屏蔽/被举报用户的交互；
- 格式错误、超大或不支持的媒体；
- 用户取消与应用前后台切换。

面向用户的错误信息必须安全、可操作，且不得暴露 Stack Trace、Secret、内部标识符或敏感数据。

10. 测试环境与数据

10.1 环境隔离

环境	目的	数据/服务
Local	开发者快速反馈与隔离测试	Fake、Mock、本地 Fixture、Emulator
CI	可重复自动化验证	临时服务/Container、Emulator、确定性 Fixture
pre/Staging	集成系统回归与 UAT	非生产项目/配置和合成账号
Production	仅限部署后 Smoke 与监控	生产服务；禁止破坏性自动测试数据

10.2 数据规则

- 使用合成身份、商品、聊天、位置、举报和交易数据。
- 禁止使用真实卡信息；使用 Stripe Test Mode 或批准的支付 Sandbox。
- 禁止将生产个人数据复制到测试环境。
- Fixture 必须具有稳定 ID 和明确的资源归属/角色。
- 测试必须自行创建或 Seed 所需状态，并在结束后清理或重置环境。
- 当时间戳和随机性影响断言时，必须使其可控。
- 媒体 Fixture 应包括有效、不支持、超大和潜在不安全样本，但不得包含真实个人信息。

10.3 自包含 E2E 设置

E2E 测试不得要求开发者记住并手工启动服务。测试套件应自行启动确定性后端/Emulator，或使用 CI 管理的 Container 和已 Seed 的 Sandbox 服务。Base URL 和 Service Adapter 必须通过依赖注入或环境配置进行切换。

11. 第三方与原生能力测试策略

依赖/能力	单元/Widget 方法	集成/E2E 方法
Firebase Auth / Google 登录	Fake Auth Repository 与受控状态	Emulator/Test Project 与受控测试身份；仅在稳定时执行原生登录 Smoke
Firestore/Firebase Rules	UI 逻辑使用 Fake Repository	基于 Emulator 的安全规则与数据集成测试
Node.js 服务/数据库	Mock Repository/Gateway	隔离测试数据库或带确定性 Seed 的 Container
Stripe/支付	Fake Gateway 覆盖全部业务状态	Test Mode；验证 Webhook 签名、幂等性、失败、取消和退款
FCM 通知	Fake Notification Gateway 与 Payload 断言	一个代表性设备/Emulator Smoke；按可行性验证前台/后台处理
相机/媒体	Fake Picker/Uploader	代表性 Emulator/Device 流程，并测试校验与上传失败
GPS/地图/安全地点	Fake Location Provider 与固定坐标	Patrol 权限允许/拒绝流程；Test Project 地图配置
AI 描述服务	确定性 Fake 响应、无效输出和超时	契约/Sandbox 检查；多数测试不得依赖实时模型输出
社交分享	Fake Share Adapter	原生分享面板 Smoke/支持平台上的人工验证

团队测试的是 KiwiShare 对这些服务的使用，而不是供应商的内部实现。

12. 非功能测试

12.1 性能

性能分析应覆盖项目 Pitch 中识别的产品风险：

- 商品列表、图片密集详情、聊天和地图的 Flutter Frame Build/Raster 表现；
- 重复选择/上传图片和长聊天会话中的内存占用；
- 商品列表/搜索延迟；
- Firestore/数据库查询与同步延迟；
- Cloud Function/API 响应时间；
- 应用启动及进入首个可用页面的时间。

性能必须在 Profile/接近 Release 的模式下测量，不能依据 Debug Mode 推断。Feature Freeze 前，团队必须记录基线，并为关键交互批准可量化预算。超过批准预算的回归必须调查，或形成明确接受的 Issue。

12.2 可靠性与恢复

验证：

- Retry/Backoff 不会产生重复商品、消息、订单、扣款或核验事件；
- 聊天及二维码/订单状态能按照规格在重启或重连后恢复；
- 临时后端失败会产生可恢复 UI 状态；
- 适用时，计划备份与恢复流程在发布前得到验证；
- `pre` 环境启用 Crash Reporting 与后端监控；
- 严重错误包含关联定位信息，但不泄露个人数据。

12.3 安全与隐私验证

安全测试由 `security-owasp.md` 管理。本策略另外要求对以下事项实施自动化回归覆盖：

- 身份认证与授权边界；
- 仅 Owner 可执行的修改；
- 仅参与者可访问的聊天；
- 仅管理员可执行的治理；
- 输入校验与净化；
- 上传类型/大小限制；
- 私密/模糊化位置处理；
- 支付 Webhook 真实性与重放防护；
- Source Code、Log、Fixture 与 Client Bundle 中不存在 Secret；
- 已实现的数据安全删除/匿名化路径。

安全敏感缺陷无论正常 UI 是否能够触达，均按 P0 处理。

13. 设备与平台覆盖

对于课程 MVP，CI 至少使用一个受支持的 Android Emulator
作为自动化原生目标，这与课程指导一致。重大演示或发布前，团队还应在至少一台有代表性的 Android 真实设备上执行 Smoke Test。

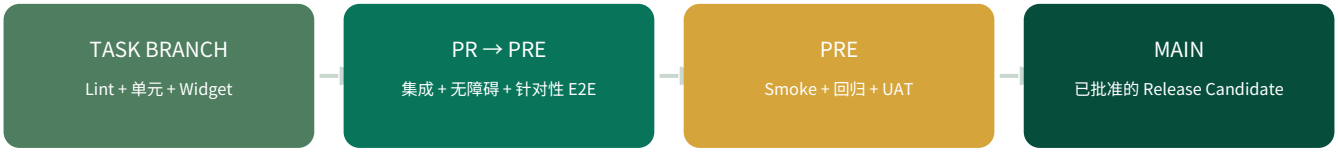
若 iOS 是已批准的发布目标，团队必须定义受支持的 iOS 版本/设备，并执行对应的
Smoke、权限、无障碍和支付测试。不能仅因 Flutter 编译成功就宣称支持 iOS。

设备矩阵必须考虑：

- 小尺寸与标准尺寸屏幕；
- 受支持的 Android/iOS 版本；
- 浅色与深色模式；
- 文字缩放；
- 无网络、慢网络与网络中断；
- 权限允许、拒绝和此前已允许；
- 应用前台、后台与恢复状态。

初始课程版本可不使用 Device Farm；若支持矩阵扩大，可在后续采用。

14. 自动化与分支质量门禁



变更越接近正式发布，Required Check 越严格

代码仓库遵循短生命周期 Task Branch，通过经过评审的 Pull Request 合并至 `pre`。测试是合并契约的一部分。

14.1 本地开发

Push 前，贡献者应针对修改的 Package 运行相关快速检查：

```
flutter analyze
flutter test
```

项目脚本确定后，等价的后端/Web Lint 与测试命令必须记录在 Package Manifest 和根 README 中。

14.2 Task Branch Push

CI 应执行：

- Format/Lint/静态分析；
- 受影响的单元测试；
- 受影响的 Widget/组件测试；
- 项目配置的 Secret Scan 与依赖检查；
- 受影响 Package 的编译/构建验证。

14.3 合并至 `pre` 的 Pull Request

Required Check 应包括：

- 全部 Task Branch Check；
- 完整 Flutter 单元/Widget 测试套件；
- 相关后端/Web 测试套件；
- API/服务集成测试；
- 发生变化的核心页面无障碍测试；
- 变更影响 P0/P1 跨系统旅程时，至少执行相关关键 E2E Smoke Test。

任何 Required Check 失败均阻止合并。Flaky Test 必须修复，或以明确负责人和 Issue 正式隔离；反复重跑 CI 直到偶然通过不可接受。

14.4 `pre` 回归

合并并部署至 `pre` 后：

- 运行集成 Smoke Suite；
- 运行定期或 Release Candidate Patrol Journey；
- 对变更区域执行针对性探索测试；
- 验证监控与 Crash Reporting；
- 对已完成产品需求执行 UAT 场景。

14.5 从 `pre` 晋级至 `main`

团队后续发布时，`pre` 到 `main` 的 PR 必须代表经过测试的 Release Candidate，并满足：

- 所有自动化 Required Check 通过；
- 完整关键旅程回归通过；
- 不存在未解决的 Release Blocking 缺陷；
- UAT/发布检查清单获得批准；
- 配置、Migration、监控、Rollback 和备份准备就绪。

任何贡献者均不得通过直接 Push 至 `pre` 或 `main` 绕过 Branch Protection。

15. 覆盖率与质量目标

以下为项目初始目标，团队确认后方可作为 CI 强制门禁：

- P0 验收标准 100% 映射到自动化测试；例外必须说明理由；
- 关键业务/Domain Logic 行覆盖率至少 80%；
- 非生成、可测试的 Flutter/后端代码整体行覆盖率至少 70%；
- 每个核心 Flutter 页面覆盖成功、加载、适用时的空状态、错误和无障碍状态；
- 覆盖矩阵包含全部 P0/P1 旅程；
- 未在 PR 中说明理由，不得降低已同意的覆盖率。

覆盖率百分比是诊断指标，而非正确性的证明。有效断言、边界情况与风险覆盖优先于为了提高数字而测试琐碎代码。生成代码与框架样板可按规定排除，但必须保持一致并形成文档。

16. 缺陷分级

严重级别	定义	示例	合并/发布规则
S0 - 严重	安全/隐私泄露、经济损失、交易状态损坏或大范围不可用	未授权数据访问、重复扣款、伪造二维码完成交易	立即停止；阻止合并和发布
S1 - 高	主要旅程被阻断且无合理替代方案	无法登录、发布、支付、聊天或完成交易	阻止受影响功能合并/发布
S2 - 中	造成明显影响但存在替代方案	某边界条件筛选错误、通知延迟但可恢复	发布前修复或明确接受
S3 - 低	外观或轻微不便	间距、非关键文案、轻微动画问题	可在记录优先级后排期

每个缺陷报告应包含：

- 环境/Build/Commit；

- 前置条件与测试账号角色；
- 可复现步骤；
- 预期结果与实际结果；
- 严重级别及受影响需求/Issue；
- 在安全前提下提供截图、录屏、Log 或 Trace；
- 回归测试要求。

17. 准入与退出标准

17.1 功能准入标准

满足以下条件后方可开始实现：

- 需求和验收标准可测试；
- Interface/Contract 与 Ownership 定义充分；
- 已识别 P0/P1 风险和必需测试层级；
- 能够创建所需 Fake、Emulator 和测试账号；
- 可能使测试设计失效的未决事项已记录。

17.2 功能 Definition of Done

满足以下条件后功能才算完成：

- 验收标准已实现；
- 已增加适当的单元、Widget、集成、无障碍和 E2E 测试；
- Happy Path 与适用失败状态通过；
- 静态分析与 Required CI Check 通过；
- 不存在 S0/S1 缺陷；
- 安全/隐私影响已评审；
- 文档与测试 Fixture 已更新；
- PR 包含测试证据并获得所需批准。

17.3 发布退出标准

仅当满足以下条件时，Release Candidate 才可晋级：

- 所有 Mandatory Suite 在目标 Release Commit 上通过；
- 全部 P0/P1 验收标准具有证据；
- 不存在未关闭的 S0/S1 缺陷；
- 已接受的 S2/S3 缺陷形成记录；
- 无障碍检查与关键人工检查通过；
- UAT Sign-off 已记录；
- 监控、备份、恢复与回滚检查完成。

18. 可追溯性、命名与证据

每项产品需求或 GitHub Issue 都应明确其验收标准和对应自动化测试。建议使用以下测试标识符：

- `UT-<MODULE>-NNN` - 单元测试；
- `WT-<MODULE>-NNN` - Flutter Widget/组件测试；
- `IT-<MODULE>-NNN` - API/服务集成测试；
- `E2E-<JOURNEY>-NNN` - Patrol/浏览器端到端测试；
- `NFT-<AREA>-NNN` - 非功能测试；
- `UAT-<JOURNEY>-NNN` - 用户验收场景。

PR 描述应报告：

- 受影响的需求/Issue；
- 新增或更新的测试层级；
- 已执行的命令/套件；
- 相关截图、录屏或性能证据；
- 已知限制或明确延后的用例。

19. 责任与维护

- 功能 Owner 与实现同步编写或更新测试。
- Reviewer 同时评审生产代码和测试质量。
- 模块 Owner 维护共享 Fixture、Fake 与 Helper。
- 除非排查证明原因另有归属，测试失败由引入该失败的变更负责人处理。
- Flaky、缓慢或过时的测试属于工程缺陷，必须跟踪。
- 共享 Fixture 必须最小、明确，且不包含个人数据或 Secret。
- 架构、发布范围、支持平台或关键第三方服务发生变化时，应评审本策略。

20. CI 强制执行前的待决事项

在将建议目标转为 Mandatory Check 前，团队必须确认：

1. 明确支持的 Android 版本，以及适用时的 iOS 版本；
2. 最终后端/Web Test Runner 与标准 Root Command；
3. Firebase Emulator 与隔离数据库策略；
4. 支付 Sandbox 与 Webhook 测试方法；
5. 哪些 Patrol Journey 每个 PR 执行，哪些仅定期或在 `pre` 执行；
6. 批准的覆盖率阈值与排除项；
7. 关键交互性能预算；
8. React 管理后台是否纳入 MVP；
9. 测试账号、Fixture 与 `pre` UAT 协调的 Owner。

在上述决定获批前，仍应按照本策略的风险和分层原则编写测试；仅数值阈值与运行门禁暂定。

21. 初始实施顺序

为确保团队能够逐步采用，按以下顺序实施：

1. 建立可注入 Repository/Gateway 和共享确定性 Fake；
2. 为身份认证、商品、订单/支付、二维码核验、聊天/举报和信用逻辑增加单元测试；
3. 为每个核心页面增加加载、成功、空、错误、交互和无障碍状态的 Widget 测试；
4. 增加隔离的 API/安全规则/数据库集成测试；
5. 为发布、权限、交易/二维码和聊天/举报建立小型 Patrol Suite；
6. 将快速检查接入每次 Task Branch Push，并将 Required Check 接入目标为 `pre` 的 PR；
7. 建立 `pre` 回归、UAT、监控与发布证据；
8. 在首个集成里程碑后，根据真实缺陷和执行时间评估测试有效性并调整。

附录 A - PR 最低测试检查清单

- ☐ 验收标准可测试且已建立链接。
- ☐ 单元测试覆盖新增或变更逻辑。
- ☐ Widget/组件测试覆盖相关 UI 状态。
- ☐ 无障碍检查覆盖变更的核心 Flutter 页面。
- ☐ API/集成测试覆盖变更的契约或持久化。
- ☐ P0/P1 失败路径与授权路径已覆盖。
- ☐ 相关 Patrol/E2E Journey 通过，或已记录延期理由。
- ☐ 测试中不包含真实 Secret、个人数据或生产 Endpoint。
- ☐ 静态分析、自动测试和受影响 Build 通过。
- ☐ 自动化无法验证的行为已附人工证据。
- ☐ 每个已修复缺陷均包含回归测试。

附录 B - 核心页面最低 Widget Contract

对每个核心 Flutter 页面，按适用情况验证：

- ☐ 渲染加载状态且测试不会 Hang；
- ☐ 渲染预期数据；
- ☐ 渲染空状态；
- ☐ 渲染安全错误，并允许 Retry/Recovery；
- ☐ 校验用户输入；
- ☐ 仅启用获授权操作；
- ☐ 完成主要交互/导航；
- ☐ 暴露有意义的语义标签；

- [ ] 符合 Android/iOS 点击目标与文字对比度规范；
- [ ] 在浅色/深色模式及代表性文字缩放下保持可读。

附录 C - 参考资料

- Andrew Meads，COMPSCI 734 Lecture 07，Testing Mobile Applications，2026。
- Five Guys，KiwiShare Elevator Pitch - Assignment 1，2026。
- Flutter 官方文档，Testing Flutter apps 与无障碍测试指南。
- Patrol 官方文档，安装与原生自动化指南。